

Week 2 — Day 5: Query Performance, Storage Layout, and Data Formats

Part 1 — Query Performance Fundamentals

Q. 1 — Full Table Scans

BritBooks has an `orders` table with one million rows.

A query searches for one customer but there is no index on `customer_id`.

Answer the following questions:

1. **What the database may need to read.** The database may read every row to find matching customers. This is called a **full table scan**.
2. **Why the query becomes slower as the table grows.** As the table grows, more rows must be checked, so the query normally takes longer.
3. **One way to improve the query.** An index on `customer_id` can help the database find matching rows without checking the entire table.

Q. 2 — Indexing Basics

BritBooks frequently runs this query:

SQL

```
SELECT order_id, amount_gbp
FROM orders
WHERE customer_id = 101;
```

1. **Which column should be indexed?** Create an index on `customer_id`.
2. **How can the index improve the query?** The index helps the database locate matching rows more directly.
3. **What is one disadvantage of adding an index?** Indexes use additional storage and add work when rows are inserted, updated, or deleted.

Q. 3 — Practical: Observe a Scan and an Index

BritBooks wants to inspect how SQLite finds orders for one customer.

REQUIRED TABLE

Column Data type

`order_id` INTEGER PRIMARY KEY

`customer_id` INTEGER NOT NULL

`amount_gbp` REAL NOT NULL

`status` TEXT NOT NULL

TASK

1. Create query_performance.db .
2. Create the orders table.
3. Insert 5,000 sample orders.
4. Use EXPLAIN QUERY PLAN before creating an index.
5. Create an index on customer_id .
6. Run EXPLAIN QUERY PLAN again.
7. Print both query plans.

STARTER CODE

P Y T H O N

```
import sqlite3

connection = sqlite3.connect("query_performance.db")
cursor = connection.cursor()

# Create and populate the table
# Check the plan before the index
# Create the index
# Check the plan after the index

connection.close()
```

Expected Observation

Before the index, the plan should contain wording similar to:

```
query_performance.py U X
Data-Engineering > Week-2 > query_performance > query_performance.py > ...
9 #1 - set up and open connection with database
10 connection = sqlite3.connect("query_performance.db")
11 cursor = connection.cursor()
12 |
13 cursor.execute("DROP TABLE IF EXISTS orders")
14
15 # 2-Create the table
16 cursor.execute("""
17 CREATE TABLE orders (
18     order_id INTEGER PRIMARY KEY,
19     customer_id INTEGER NOT NULL,
20     amount_gbp REAL NOT NULL,
21     status TEXT NOT NULL
22 )
23 """)
24
25 #...and populate the table
26 orders = [
27     (
28         order_id,
29         ((order_id - 1) % 500) + 1,
30         10 + (order_id % 90),
31         "Dispatched" if order_id % 2 == 0 else "Confirmed"
32     )
33     for order_id in range(1, 5001)
34 ]
35
36 print(orders[:5])
37
38 #3-Insert all orders
39 cursor.executemany("""
40 INSERT INTO orders (order_id, customer_id, amount_gbp, status)
41 VALUES (?, ?, ?, ?)
42 """, orders)
43
44 connection.commit()
45 query = ""

query_performance.py U X
Data-Engineering > Week-2 > query_performance > query_performance.py > ...
44 connection.commit()
45 query = ""
46 SELECT order_id, amount_gbp
47 FROM orders
48 WHERE customer_id = 101
49 ""
50
51 # 4.Check the plan before the index
52 cursor.execute(f"EXPLAIN QUERY PLAN {query}")
53 print("Before index:")
54 for row in cursor.fetchall():
55     print(row)
56
57 # 5-Create the index
58 cursor.execute("""
59 CREATE INDEX idx_orders_customer_id
60 ON orders(customer_id)
61 """)
62
63 connection.commit()
64
65 # 6Check the plan after the index
66 cursor.execute(f"EXPLAIN QUERY PLAN {query}")
67 print("\nAfter index:")
68 for row in cursor.fetchall():
69     print(row)
70
71 connection.close()
72
73
```

Q. 4 — Practical: Filter Early

BritBooks needs the total value of dispatched orders.

TASK

Create an SQLite script that compares these approaches:

Approach A

- Read every order.
- Filter `Dispatched` rows in Python.
- Calculate the total in Python.

Approach B

- Filter rows in SQL.
- Calculate the total using `SUM`.

- Return only the final value.

Print both totals and confirm that they match.

SOLUTION

PYTHON

```
import sqlite3

connection = sqlite3.connect("filter_early.db")
cursor = connection.cursor()

cursor.execute("DROP TABLE IF EXISTS orders")

cursor.execute("""
CREATE TABLE orders (
order_id INTEGER PRIMARY KEY,
amount_gbp REAL NOT NULL,
status TEXT NOT NULL
)
""")

orders = [
(1, 34.99, "Dispatched"),
(2, 18.50, "Confirmed"),
(3, 42.75, "Dispatched"),
(4, 12.00, "Cancelled")
]

cursor.executemany(
"INSERT INTO orders VALUES (?, ?, ?)",
orders
)
connection.commit()

# Approach A: read everything and filter in Python
cursor.execute("SELECT * FROM orders")
all_orders = cursor.fetchall()

python_total = sum(
amount
for _, amount, status in all_orders
if status == "Dispatched"
)

# Approach B: filter and aggregate in SQL
cursor.execute("""
SELECT SUM(amount_gbp)
FROM orders
WHERE status = 'Dispatched'
""")
sql_total = cursor.fetchone()[0]

print(f"Python total: £{python_total:.2f}")
print(f"SQL total: £{sql_total:.2f}")
print(f"Totals match: {python_total == sql_total}")

connection.close()
```

Expected Output

TEXT

```
Python total: £77.74
SQL total: £77.74
Totals match: True
```

Key Point

Filtering and aggregating in the database usually avoids transferring unnecessary rows and columns.

What happens during a full table scan?

- A. The database reads only one indexed row.
- B. The database checks every row in the table.
- C. The database deletes unused rows.
- D. The database compresses the table

Q. 6 — MCQ: Index Choice

Which column is the best index candidate for this common query?

SQL

```
SELECT *  
FROM orders  
WHERE customer_id = 101;
```

- A. customer_id
- B. amount_gbp
- C. A column that is never queried
- D. Every column automatically

Q. 7 — MCQ: Index Trade-off

What is a common disadvantage of an index?

- A. It prevents all queries.
- B. It uses storage and adds write overhead.
- C. It removes primary keys.
- D. It converts rows into files.

Q. 8 — MCQ: Filtering Early

Which query transfers the least unnecessary data?

- A. SELECT * FROM orders;
- B. SELECT * FROM orders; followed by Python filtering
- C. SELECT order_id FROM orders WHERE status = 'Dispatched';
- D. SELECT * FROM customers, orders;

Q. 9 — MCQ: Low-Cardinality Columns

An `order_status` column contains only `Confirmed`, `Dispatched`, and `Cancelled`. Why might an index on this column provide limited improvement for some queries?

- A. The column has very few distinct values.
- B. Text columns can never be indexed.
- C. Indexes work only on dates.
- D. Status values are stored in pounds.

Part 2 — Storage Layout and Partitioning

Q. 10 — Partitioning Fundamentals

QUESTION

BritBooks stores three years of order data.

Most reports filter by order date.

Explain:

1. A suitable partition key

A suitable key is the order date, usually split into `year` and `month`.

2. A possible folder layout:

```
/orders/year=2026/month=01/  
/orders/year=2026/month=02/
```

3. How partition pruning reduces the amount of data read

A query for February 2026 can read only the matching folder instead of scanning every year and month.

Q. 11 — Design a Partition Scheme

SCENARIO

BritBooks has:

- 100 million orders per year
- Reports that normally filter by month
- Very few reports for a single hour

QUESTION

Choose one layout and justify your decision:

Option A

/orders/year=2026/month=07/

Option B

/orders/year=2026/month=07/day=17/hour=10/

Option A is more suitable.

Monthly filtering matches the normal query pattern. Hourly partitions would create many directories and could produce very small partitions.

Q. 12 — The Small-Files Problem

QUESTION

A daily partition contains 10,000 files, each approximately 50 KB.

Explain:

- Why opening many small files can be inefficient
- What compaction means
- When a compaction job could run

Each file has opening and metadata overhead. Thousands of files can therefore take longer to list, open, and schedule than a smaller number of larger files.

Compaction combines many small files into fewer larger files.

BritBooks could run compaction after the daily load or during a quieter period.

Q. 13 — Practical: Combine Small CSV Files

BritBooks receives three small order files for one day.

INPUT FILES

TEXT

orders_1.csv
orders_2.csv
orders_3.csv

Each file contains: **order_id,customer_name,amount_gbp**

TASK

- Create a folder named `small_order_files` .
- Create the three CSV files with one order in each.
- Combine them into `orders_2026-07-17.csv` .
- Write the header only once.
- Print the number of combined rows.

SOLUTION

P Y T H O N

```
from pathlib import Path
import csv

input_folder = Path("small_order_files")
input_folder.mkdir(exist_ok=True)

files_and_rows = {
    "orders_1.csv": [1001, "Amelia Clarke", 34.99],
    "orders_2.csv": [1002, "Oliver Bennett", 18.50],
    "orders_3.csv": [1003, "Aisha Khan", 42.75]
}

header = ["order_id", "customer_name", "amount_gbp"]

for file_name, row in files_and_rows.items():
    with (input_folder / file_name).open(
        "w",
        newline="",
        encoding="utf-8"
    ) as file:
        writer = csv.writer(file)
        writer.writerow(header)
        writer.writerow(row)

output_file = Path("orders_2026-07-17.csv")
combined_rows = []

for input_file in sorted(input_folder.glob("orders_*.csv")):
    with input_file.open(
        "r",
        newline="",
        encoding="utf-8"
    ) as file:
        reader = csv.DictReader(file)
        combined_rows.extend(reader)

with output_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.DictWriter(file, fieldnames=header)
    writer.writeheader()
    writer.writerows(combined_rows)

print(f"Combined rows: {len(combined_rows)}")
print(f"Output file: {output_file}")
```


Expected Output

TEXT

Combined rows: 3
Output file: orders_2026-07-17.csv

Key Point

Real data platforms perform the same general operation at a larger scale and often write the result in Parquet or ORC.

Q. 14 — MCQ: Partition Key

Which is the best partition key when most queries filter orders by month?

- A. Customer name
- B. Order month
- C. Product description
- D. Order amount rounded to the nearest pound

Q. 15 — MCQ: Partition Pruning

What does partition pruning do?

- A. Deletes old partitions automatically
- B. Reads only partitions that match the filter
- C. Creates an index on every column
- D. Converts CSV into JSON

Q. 16 — MCQ: Over-Partitioning

What is a possible result of partitioning data too finely?

- A. Too many small folders and files
- B. All rows become duplicates
- C. SQL stops supporting filters
- D. Compression becomes impossible

Q. 17 — MCQ: Small Files

Why are thousands of tiny files often inefficient?

- A. They cannot contain numbers.
- B. Each file adds metadata and opening overhead.
- C. They always delete their schemas.
- D. Small files cannot be stored in folders.

Q. 18 — MCQ: Compaction

What is compaction?

- A. Splitting one file into thousands of files
- B. Combining small files into fewer larger files
- C. Removing all partition columns
- D. Replacing SQL with Python

Part 3 — File Formats and Columnar Storage

Q. 19 — Compare Common File Formats

QUESTION

Complete the table using short phrases.

Format	Text or binary?	Row or column oriented?	Common use
--------	-----------------	-------------------------	------------

CSV	Row-like records	Simple exchange and manual inspection
-----	------------------	---------------------------------------

Parquet	Binary	Columnar	Data lake and analytics workloads
---------	--------	----------	-----------------------------------

Avro	Binary	Row-oriented	Data exchange and streaming events
------	--------	--------------	------------------------------------

ORC	Binary	Columnar	Hive and Hadoop analytics
-----	--------	----------	---------------------------

Q. 20 — Row and Columnar Storage

QUESTION

BritBooks has an analytics file with 80 columns.

A report needs only:

```
      id
• • • amount_
order_d gbp
ate
product_
```

Explain why a columnar format may read less data than CSV.

CSV stores complete rows, so a reader normally processes data from every column in each row.

A columnar format stores values from the same column together. An analytics engine can therefore read only the three required columns.

Q. 21 — Choose a File Format

QUESTION

Choose the most suitable option and give one reason.

Requirement Choose from A small file that a non-technical user can open easily CSV / Parquet Large

analytical data where queries select a few columns CSV / Parquet Streaming events with an explicit schema

Avro / CSV

Large Hive analytics tables ORC / CSV

Requirement Choice Reason:

Easy manual opening CSV It is plain text and widely supported.

Large analytical data Parquet It is columnar and supports compression

Streaming events with schema Avro, it stores structured records with a defined schema

Hive analytics tables ORC It is a columnar format designed for large analytical workloads.

Q. 22 — Practical: Create an Avro-Style Schema File

BritBooks sends order events between systems.

REQUIRED FIELDS

Field Type

order_id string

customer_id string

amount_gbp double

order_status string

coupon_code null or string, default null

TASK

- Create the schema as a Python dictionary.
- Write it to `order_event_schema.json`.
- Use indentation to keep the JSON readable.
- Read the file again.

- Print the field names.

SOLUTION

P Y T H O N

```
import json

schema = {
    "type": "record",
    "name": "OrderEvent",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "customer_id", "type": "string"},
        {"name": "amount_gbp", "type": "double"},
        {"name": "order_status", "type": "string"},
        {
            "name": "coupon_code",
            "type": ["null", "string"],
            "default": None
        }
    ]
}

with open(
    "order_event_schema.json",
    "w",
    encoding="utf-8"
) as file:
    json.dump(schema, file, indent=2)

with open(
    "order_event_schema.json",
    "r",

    encoding="utf-8"
) as file:
    loaded_schema = json.load(file)

field_names = [
    field["name"]
    for field in loaded_schema["fields"]
]

print(field_names)
```

Expected Output

T E X T

```
['order_id', 'customer_id', 'amount_gbp', 'order_status', 'coupon_code']
```

Key Point

The optional field has a default value, which helps older records remain readable when a new field is introduced.

Q. 23 — MCQ: CSV ^{M C Q}

Which statement about CSV is correct?

- A. It always stores a strict schema.
- B. It is a human-readable text format.**
- C. It is a columnar binary format.

D. It automatically supports nested objects.

Q. 24 — MCQ: Parquet

Why is Parquet commonly used for analytics?

A. It stores every value as plain text.

B. It is columnar and can read selected columns.

C. It supports only one data type.

D. It cannot be compressed.

Q. 25 — MCQ: Avro

Which use case best matches Avro?

A. A simple spreadsheet opened manually

B. Structured event exchange with schema evolution

C. Image storage

D. Graph traversal

Q. 26 — MCQ: ORC

ORC is mainly associated with:

A. Columnar analytics in the Hadoop and Hive ecosystem

B. Storing passwords

C. Editing source code

D. Relational transactions

Q. 27 — MCQ: Compression

What is the main benefit of compression?

A. It always removes CPU usage.

B. It reduces storage size and data read from disk. > Compression saves storage and input/output, although decompression uses some CPU.

C. It removes the need for a schema.

D. It converts columns into rows.

Part 4 — Combined Design Decisions

Q. 28 — Identify the Requirements

SCENARIO

BritBooks needs a daily sales report.

The report:

- Filters by order month
- Groups by product
- Reads only order date, product ID, quantity, and amount
- Uses data stored for five years

QUESTION

Identify:

1. The best partition field
Partition using order date, such as year and month.
2. A suitable analytical file format
Use Parquet or ORC for analytical storage.
3. The columns the query should select
Select only order date, product ID, quantity, and amount.
4. One reason to avoid `SELECT *`
Select `*` will pull ALL data and that can be a time and resources consuming task.
`SELECT *` reads unnecessary columns and increases input/output.

Q. 29 — Create a Storage Decision Table

SCENARIO

BritBooks has these datasets:

- Raw order exports
- Cleaned analytical orders
- Streaming click events
- Monthly sales summaries

TASK

Create a table with:

Dataset File format Partition key Reason Use CSV, Parquet, Avro, or ORC where appropriate.

Dataset File format Partition key Reason

Raw order exports	CSV	Ingestion date	Keeps the original portable source files	
			Parquet	Order year and month pruning
Cleaned analytical orders			Supports columnar analytics and	
			Avro	Event date and hour Supports structured event records and schema evolution
Streaming click events				
		summaries	Summary year and	Supports fast analytical
Monthly sales		Parquet or ORC	month	reads

Q. 30 — Practical: Create a Partitioned Folder Layout

BritBooks wants to organise orders by year and month.

DATA

1001, 2026-06-28, £34.99
1002, 2026-07-02, £18.50
1003, 2026-07-17, £42.75

TASK

- Create an `orders_lake` folder.
- Create `year=YYYY/month=MM` folders from each date.
- Write each order to an `orders.csv` file in the correct partition.
- Include the CSV header.
- Print the created file paths.

SOLUTION

PYTHON

```
from pathlib import Path
from datetime import datetime
import csv
```

```
orders = [
    {
        "order_id": 1001,
        "order_date": "2026-06-28",
        "amount_gbp": 34.99
    },
    {
        "order_id": 1002,
        "order_date": "2026-07-02",
        "amount_gbp": 18.50
    }
]
```

```

    },
    {
        "order_id": 1003,
        "order_date": "2026-07-17",
        "amount_gbp": 42.75
    }
]

base_folder = Path("orders_lake")
header = ["order_id", "order_date", "amount_gbp"]

for order in orders:
    order_date = datetime.strptime(
        order["order_date"],
        "%Y-%m-%d"
    )
    partition_folder = (
        base_folder
        / f"year={order_date.year}"
        / f"month={order_date.month:02d}"
    )
    partition_folder.mkdir(parents=True, exist_ok=True)

    output_file = partition_folder / "orders.csv"
    file_exists = output_file.exists()

    with output_file.open(
        "a",
        newline="",
        encoding="utf-8"
    ) as file:
        writer = csv.DictWriter(file, fieldnames=header)

        if not file_exists:
            writer.writeheader()

        writer.writerow(order)

for file_path in sorted(base_folder.rglob("orders.csv")):
    print(file_path)

```

Expected Folder Layout

T E X T

```

orders_lake/
├── year=2026/
│   ├── month=06/
│   │   └── orders.csv
│   └── month=07/
│       └── orders.csv

```

Key Point

Queries filtering on year and month can avoid reading unrelated partitions.

Q. 31 — Review an Analytics Design

PROPOSED DESIGN

BritBooks stores:

- Five years of orders in one uncompressed CSV file
- 100 columns in every row
- Reports that normally use four columns
- No date partitions
- One daily process that appends thousands of tiny files

QUESTION

Recommend four improvements.

ANSWER

- Convert cleaned analytics data to Parquet or ORC.
- Partition data by a commonly filtered date field.
- Select only required columns in queries.
- Compact the daily small files into fewer larger files.
- Use compression appropriate for the chosen analytical format.

Q. 32 — MCQ: Analytics Format

Which format is the strongest default choice for large analytical order data?

- A. Parquet
- B. Plain text without separators
- C. Image files
- D. HTML

Q. 33 — MCQ: Partition Filter

Which filter is most likely to support pruning for this layout?

/orders/year=2026/month=07/

- A. WHERE customer_name = 'Amelia Clarke'
- B. WHERE year = 2026 AND month = 7
- C. WHERE amount_gbp > 20
- D. WHERE status = 'Confirmed'

Q. 34 — MCQ: Projection

What does projection mean in query optimisation?

- A. Selecting only required columns
- B. Deleting old rows
- C. Creating additional small files
- D. Sorting every table

Q. 35 — MCQ: Streaming Events ^{MCQ}

Which format is a suitable choice for structured streaming event records?

- A. Avro
- B. Bitmap image
- C. PDF
- D. Plain HTML

Q. 36 — MCQ: Combined Optimisation ^{MCQ}

Which combination is most suitable for monthly sales analytics?

- A. One large CSV, `SELECT *`, and no date filter
- B. Parquet, date partitions, early filtering, and required columns only
- C. Thousands of tiny text files and no compression
- D. Graph storage for every sales total

Final Recap

Before finishing, confirm that you can:

- Explain a full table scan.
- Describe how an index can improve a query.
- Explain why indexes also have a cost.
- Filter rows early and select required columns.
- Design a basic date partition layout.
- Explain partition pruning.
- Describe the small-files problem and compaction.
- Compare CSV, Parquet, Avro, and ORC.
- Explain why columnar formats help analytics.
- Choose a format and layout from business requirements.

Practice Reminder

- Use small datasets while learning.
- Query timings can differ between computers.
-

Focus on query plans and data-access patterns rather than exact timings.

- Do not add partitions or indexes without considering how the data is queried.